Ще започнем с пример, който припомня как се образуват множествата Gen и $Reach$ от предходната лекция.

Пример. Нека е дадена следната граматика G:

$$S \to AB \mid ab, \;\; A \to aA \mid \epsilon, \;\; B \to bB.$$

Изпълняваме първо алгоритъма за Gen:

$$Gen[0] = \emptyset, \;\; Gen[1] = \{S, A\}, \;\; Gen[2] = \{S, A\}.$$

На първата стъпка добавяме S и A заради правилата $S \to ab$ и $A \to \epsilon$. Променливата B не влиза на втората стъпка, тъй като няма правило с лява част B и дясна част от $(\Sigma \cup \{S, A\})^\star$. Така алгоритъмът приключва с резултат $Gen = \{S, A\}$. Новата граматика G' се образува като запазим само правилата, в които участват S и A:

$$S \to ab, \;\; A \to aA \mid \epsilon.$$

Върху получената граматика G' изпълняваме алгоритъма за $Reach$:

$$Reach[0] = \{S\}, \;\; Reach[1] = \{S\}.$$

Няма правила с лява част S, които да съдържат променливи в дясната част. Затова алгоритъмът приключва с резултат $Reach = \{S\}$. В новата граматика G'' запазваме правилата, в които участва само S:

$$S \to ab.$$

Забележете, че редът на прилагане на Gen и $Reach$ е съществен. Да започнем първо с $Reach$:

$$Reach[0] = \{S\}, \;\; Reach[1] = \{S, A, B\}, \;\; Reach[2] = \{S, A, B\}.$$

Променливите A и B влизат на първата стъпка заради правилото $S \to AB$. Тъй като няма други променливи, $Reach = \{S, A, B\}$. Естествено, трябва да запазим всички правила. След това прилагаме Gen и резултатът е както по-горе, т.е. новата граматика има правилата $S \to ab, \;\; A \to aA \mid \epsilon$. В нея A е недостижима променлива (и значи не е полезна).

Този пример показва, че след изпълняване на Gen една достижима променлива може да стане недостижима. От друга страна, след изпълнение на $Reach$ не е възможно генерираща променлива да стане негенерираща. В случай, че A е достижима променлива, всички променливи участващи в извода $A \stackrel{\star}{\Rightarrow} \gamma$ също ще бъдат достижими.

Твърдение 1 *За всяка безконтекстна граматика $G = (V, \Sigma, R, S)$ с $\mathcal{L}(G) \neq \emptyset$, съществува безконтекстна граматика $G' = (V', \Sigma, R', S)$, такава че $\mathcal{L}(G') = \mathcal{L}(G)$ и всяка променлива в G' е полезна. Също така, G' може да се построи ефективно от G.*

Доказателство. При дадена G прилагаме първо Gen и след това $Reach$.

Премахване на ϵ-правила

Правилата от вида $A \to \epsilon$ наричаме *ϵ-правила*. Следващото твърдение показва, че можем да ги елиминираме от граматиката, с цената на това, че трябва да премахнем празната дума от езика.

Твърдение 2 *За всяка безконтекстна граматика $G = (V, \Sigma, R, S)$ съществува безконтекстна граматика $G' = (V, \Sigma, R', S)$, в която няма ϵ-правила и също така $\mathcal{L}(G') = \mathcal{L}(G) \setminus \{\epsilon\}$.*

Доказателство. Първо ще си поставим за цел да генерираме всички променливи A, които извеждат ϵ. Нека

$$E \;=\; \{A \in V \mid A \overset{\star}{\lhd} \epsilon\}.$$

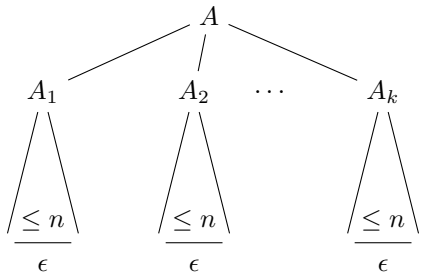
Отново ще пресмятаме E на стъпки:

$$E[n] \;=\; \{A \in V \mid A \overset{\leq n}{\lhd} \epsilon\}.$$

В началото $E[0] \;=\; \emptyset$ и след това

$$E[n+1] \;=\; \{A \in V \mid \exists \alpha \in E[n]^\star \; A \to_G \alpha\}.$$

Ситуацията е много подобна с алгоритъма за Gen: ($\alpha = A_1 A_2 \ldots A_k$)



Разбира се, букви от Σ не могат да участват в изводи на празната дума. Да напомним, че $\emptyset^\star = \{\epsilon\}$ и затова $E[1] = \{A \in V \mid A \to_G \epsilon\}$, т.е. на първата стъпка трябва да вземем променливите от левите части на ϵ-правилата в граматиката. Обръщаме специално внимание, че на следващи стъпки в E могат да влязат и други променливи (които не са лява част на ϵ-правило).

Тъй като променливите са краен брой, ще настъпи момент n, при който $E[n] = E[n+1]$ и тогава множество E съвпада с това последно $E[n]$.

За да получим новата граматика G' ще трябва да предвидим, за всяко участие на променлива $X \in E$ в дясна част на правило, възможност X да бъде изтрита от тази дясна част. При това, ако в някоя дясна част на правило участват повече от една променливи от E, то трябва да се предвидят всички възможни начини за изтриване.

По-точно, в новата граматика G' трябва да включим следните правила:

$$R' = \{A \to Y_1 Y_2 \dots Y_k \mid \exists A \to_G X_1 X_2 \dots X_k,\ X_i = Y_i \text{ или } (X_i \in E \ \&\ Y_i = \epsilon)\}.$$

Поне едно Y_i трябва да е различно от ϵ, тъй като целим да премахнем ϵ-правилата. В най-лошия случай, $X_i \in E$ за всяко i и така заради правилото $A \to_G X_1 X_2 \dots X_k$ трябва да добавим $2^k - 1$ правила в R'. Това показва, че алгоритъмът в общия случай е експоненциален. Не е трудно да се съобрази, че $\mathcal{L}(G') = \mathcal{L}(G) \setminus \{\epsilon\}$, тъй като дърветата на извод съвместими с G' съвпадат с дърветата на извод съвместими G след премахване на всички поддървета, извеждащи ϵ.

Пример. Да предположим, че $B, D \in E,\ C \notin E$.

1) На правилото $A \to aBCD$ в G съответстват 4 правила в G':

$$A \to aBCD \mid aCD \mid aBC \mid aC.$$

2) На правилото $A \to BD$ в G съответстват 3 правила в G':

$$A \to BD \mid B \mid D.$$

Премахване на преименуващите правила

Правилата от вида $A \to B$, където $A, B \in V$ наричаме *преименуващи*. Нашата цел ще е да ги елиминираме от граматиката.

Твърдение 3 *За всяка безконтекстна граматика $G = (V, \Sigma, R, S)$ без ϵ-правила, съществува безконтекстна граматика $G' = (V, \Sigma, R', S)$, в която няма преименуващи правила и $\mathcal{L}(G') = \mathcal{L}(G)$.*

Доказателство. Ще генерираме всички двойки от променливи, такива че $A \overset{\star}{\Rightarrow}_G B$. Забележете, че в G няма ϵ-правила и затова всеки извод $A \overset{\star}{\Rightarrow}_G B$ всъщност има вида $A \Rightarrow_G C_1 \Rightarrow_G C_2 \Rightarrow_G \dots \Rightarrow_G C_n \Rightarrow_G B$. При наличието на ϵ-правила в такъв извод може да участва и дума с дължина повече от 1 (например, $A \Rightarrow_G BC \Rightarrow_G B$, ако има правило $C \to_G \epsilon$).

Дефинираме множеството

$$Ren \ = \ \{(A, B) \in V \times V \mid A \overset{\star}{\Rightarrow}_G B\}$$

и го пресмятаме на стъпки:

$$Ren[n] \ = \ \{(A, B) \in V \times V \mid A \overset{\leq n}{\Rightarrow}_G B\}.$$

Тук няма особена разлика дали разглеждаме дърво на извод или линеен извод (изглеждат еднакво).

Започваме с $Ren[0] = \{(A, A) \mid A \in V\}$ и след това

$$Ren[n+1] = Ren[n] \cup \{(A, B) \mid \exists C \in V \;\; A \to_G C \;\&\; (C, B) \in Ren[n]\}.$$

Налага се да добавим $Ren[n]$, ако искаме растяща верига, тъй като $Ren[0]$ не се съдържа автоматично в $Ren[1]$. Спираме на първото n, такова че $Ren[n] = Ren[n+1]$ и резултатът Ren съвпада с последното $Ren[n]$.

Използвайки изучавани понятия можем да кажем, че Ren е рефлексивното и транзитивно затваряне на релацията $\{(A, B) \mid A \to_G B\}$. Също можем да си мислим, че тази релация представя ориентиран граф и тогава $(A, B) \in Ren$ точно когато има маршрут с начало A и край B.

Остава да покажем как се образуват правилата в новата граматика G':

$$R' = \{A \to \alpha \mid \exists B \in V \; (A, B) \in Ren \;\; \& \;\; B \to_G \alpha \;\; \& \;\; \alpha \notin V\}.$$

Разбира се, $\alpha \notin V$ означава, че в R' не допускаме преименуващи правила. Можем да обобщим идеята по следния начин. Бихме искали изводът

$$A \Rightarrow_G C_1 \Rightarrow_G C_2 \Rightarrow_G \; \ldots \; \Rightarrow_G C_n \Rightarrow_G B \Rightarrow_G \beta$$

в G да се симулира в новата граматика на една стъпка: $A \Rightarrow_{G'} \beta$ и затова добавяме правилото $A \to_{G'} \beta$ на основание на това, че $(A, B) \in Ren$, т.е. има последователност от преименуващи правила с начало A и край B и $B \to_G \beta$ е правило в G.

Премахване на дългите правила

Правилата от вида $A \to \beta$, където $|\beta| \geq 3$ наричаме *дълги правила*.

Нека $A \to X_1 X_2 \ldots X_k$ е дълго правило, $k \geq 3$. Можем да го заменим със следните правила:

$$A \to X_1 A_1, \;\; A_1 \to X_2 A_2, \;\; \ldots, \;\; A_{k-2} \to X_{k-1} X_k,$$

където $A_1, A_2, \ldots, A_{k-2}$ са нови променливи. Разбира се, за различните дълги правила ще използваме различни нови променливи.

Нормална форма на Чомски

Безконтекстната граматика G е в *нормална форма на Чомски*, ако правилата в G са от вида $A \to BC$ или $A \to a$, където $A, B, C \in V$, $a \in \Sigma$.

Теорема 4 *За всеки безконтекстен език $L = \mathcal{L}(G)$, такъв че $\epsilon \notin L$ съществува безконтекстна граматика G' в нормална форма на Чомски, такава че $\mathcal{L}(G') = L$. При това, G' може да се построи ефективно от G.*

Доказателство. Започваме с безконтекстна граматика G и върху нея извършваме опростяванията в следния ред:

1. Премахваме дългите правила.
2. Премахваме ϵ-правилата.
3. Премахваме преименуващите правила.
4. Премахваме безполезните променливи (не е задължително).

Това че премахваме дългите правила преди алгоритъма за ϵ-правилата гарантира, че няма да се получи експоненциалното добавяне на нови правила, което споменахме по-горе. На всяко старо правило ще съответстват най-много 3 нови правила.

Съществено е да премахнем ϵ-правилата преди преименуващите правила, тъй като стана ясно, че използваният алгоритъм разчита на това. Обърнете внимание също, че е възможно в G да няма преименуващи правила, но след премахването на ϵ-правилата да се появят такива.

След изпълняване на опростяванията получените правила имат следния вид: $A \to a,\ A \to BC,\ A \to bX,\ A \to Yc,\ A \to bc$.

За да премахнем последните три вида правила избираме по една нова променлива U за всяка буква $u \in \Sigma$ и добавяме правилото $U \to u$. След това заместваме правилата $A \to bX,\ A \to Yc,\ A \to bc$ съответно с $A \to BX,\ A \to YC,\ A \to BC$. Така получената граматика вече е в нормална форма на Чомски.

В случай, че искаме $\epsilon \in L$, то добавяме нова начална променлива S_0 и правилата $S_0 \to \epsilon$, $S_0 \to \alpha$ за всяко правило $S \to_G \alpha$. По този начин запазваме ϵ, но правилото $S_0 \to \epsilon$ не може да участва в други изводи.

Непосредствената полза от нормалната форма на Чомски ще бъде следния ефективен алгоритъм за проблема за принадлежност.

CYK алгоритъм

Дадена е $G = (V, \Sigma, R, S)$ в нормална форма на Чомски и дума $\alpha \in \Sigma^\star$, $\alpha = a_0 a_1 \dots a_{n-1}$. Въпросът, който искаме да разрешим е дали $\alpha \in \mathcal{L}(G)$?

Идеята на CYK алгоритъма е да потърсим отговора на по-общия въпрос дали $A \stackrel{\star}{\Rightarrow}_G a_i a_{i+1} \dots a_{i+s}$ за $A \in V$ и $i \leq i + s < n$ и затова дефинираме

$$V[i][i+s] \ = \ \{A \in V \mid A \stackrel{\star}{\Rightarrow}_G a_i a_{i+1} \dots a_{i+s}\}.$$

При $s = 0$ получаваме $V[i][i] \ = \ \{A \in V \mid A \to_G a_i\}$ (няма друг начин да се изведе a_i от A, тъй като G е в нормална форма на Чомски).

При $s > 0$ основната идея се корени в следното наблюдение.

Лема 5

$$A \stackrel{\star}{\Rightarrow}_G a_i a_{i+1} \dots a_{i+s} \iff \exists B, C \in V \ \& \ \exists k \in \mathbb{N},\ i \leq k < i + s,$$
$$A \to_G BC \ \& \ B \stackrel{\star}{\Rightarrow}_G a_i a_{i+1} \dots a_k \ \& \ C \stackrel{\star}{\Rightarrow}_G a_{k+1} \dots a_{i+s}.$$

Доказателство. Нека $s > 0$ и $A \stackrel{\star}{\Rightarrow}_G a_i a_{i+1} \dots a_{i+s}$. Тъй като G е в нормална форма на Чомски, първото използвано правило в този извод трябва да има вида $A \to_G BC$ за някои променливи B, C. Така получаваме $BC \stackrel{\star}{\Rightarrow}_G a_i a_{i+1} \dots a_{i+s}$. От основното твърдение за безконтекстни граматики съществува разбиване на $a_i a_{i+1} \dots a_{i+s}$ на две части $a_i a_{i+1} \dots a_k$ и $a_{k+1} \dots a_{i+s}$, за които имаме $B \stackrel{\star}{\Rightarrow}_G a_i a_{i+1} \dots a_k$ и $C \stackrel{\star}{\Rightarrow}_G a_{k+1} \dots a_{i+s}$. При това, $i \leq k < i + s$, тъй като тези две части трябва да са непразни думи (отново защото G е в нормална форма на Чомски).

Обратно, нека е изпълнено условието вдясно на еквивалентността. Тогава можем да получим извода $A \Rightarrow_G BC \stackrel{\star}{\Rightarrow}_G a_i a_{i+1} \dots a_k a_{k+1} \dots a_{i+s}$.

Лемата показва, че въпросът за думата $a_i a_{i+1} \dots a_{i+s}$ се свежда към същия въпрос за по-късите думи $a_i a_{i+1} \dots a_k$ и $a_{k+1} \dots a_{i+s}$. В това е същността на техниката, която се нарича динамично оптимиране. Започваме със $s = 0$ и постепенно пресмятаме множествата $V[i][i + s]$ за думи с все по-голяма дължина.

Прилагаме псевдокод на алгоритъма:
(входът е (G, α), където $\alpha = a_0 a_1 \dots a_{n-1}$)
for all i with $i < n$
  $V[i][i] \ = \ \{A \in V \mid A \to_G a_i\}$;
for all i, j with $i < j < n$
  $V[i][j] \ = \ \emptyset$;
for all s with $1 \leq s < n$
  for all i with $0 \leq i < n - s$
    for all k with $i \leq k < i + s$
      if $\exists A \to_G BC \ \ \& \ \ B \in V[i][k] \ \ \& \ \ C \in V[k + 1][i + s]$,
        $V[i][i + s] \ = \ V[i][i + s] \cup \{A\}$;
if $S \in V[0][n - 1]$, return $\alpha \in \mathcal{L}(G)$;
else return $\alpha \notin \mathcal{L}(G)$.

Наличието на три вложени цикъла води до наблюдението, че сложността на алгоритъма е $O(n^3)$.

Недетерминирани стекови автомати

За разлика от обикновените автомати, които познаваме, стековите автомати разполагат с допълнителна памет, организирана като *стек*.

Съдържанието на стека разглеждаме като дума над стековата азбука Γ. Първата ѝ буква наричаме *връх* на стека, а последната – *дъно* на стека.

Характерното за стека е, че прочитането (*peek*), добавянето (*push*) и изтриването (*pop*) на букви се извършва само в неговия връх.

За да опростим описанието на работата на стека ще допуснем единствена основна операция: замяна на върха на стека $A \in \Gamma$ с друга дума $\beta \in \Gamma^\star$.

По този начин, ако $A\gamma$ е текущото съдържание на стека, то новото съдържание е $\beta\gamma$.

Операциите *peek*, *push* и *pop* се моделират с основната операция по следния начин:

1. *peek* (прочитане на върха на стека) се получава при $\beta = A$;
2. *push* B (добавяне на $B \in \Gamma$ към стека) се получава при $\beta = BA$;
3. *pop* (изтриване на върха на стека) се получава при $\beta = \epsilon$.

Обратно, чрез един *pop* и последователност от няколко *push* можем да симулираме основната операция. Например, ако $\beta = ABC$, то замяната на върха на стека с β се симулира с последователните операции

$$pop,\ push\ C,\ push\ B,\ push\ A$$

(обърнете внимание на преобръщането).

Дефиниция 6 *Недетерминиран стеков автомат е седморка от вида*

$$P = \langle Q, \Sigma, \Gamma, \sharp, \Delta, q_{start}, q_{accept} \rangle,$$

където:

- Q *е крайно множество от състояния;*
- Σ *е крайна входна азбука;*
- Γ *е крайна стекова азбука;*
- $\sharp \in \Gamma$ *е символ за дъно на стека;*
- $\Delta : Q \times \Sigma_\epsilon \times \Gamma \to \mathcal{P}(Q \times \Gamma^{\leq 2})$ *е функция на преходите;*
- $q_{start} \in Q$ *е начално състояние;*
- $q_{accept} \in Q$ *е заключително състояние.*

Използвали сме означенията $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$ *и* $\Gamma^{\leq 2} = \{\epsilon\} \cup \Gamma \cup \Gamma^2$.

За специалния символ $\sharp \in \Gamma$ приемаме, че ще го поставяме само в дъното на стека. Чрез него ще можем да разпознаваме дали стекът съдържа други букви. Необходимостта от използване на $\sharp$ произлиза от това, че ако съдържанието на стека е ϵ, то изчислението на автомата P винаги ще спира.

Забележете типа на функцията на преходите $\Delta : Q \times \Sigma_\epsilon \times \Gamma \to \mathcal{P}(Q \times \Gamma^{\leq 2})$. По дадено състояние $q \in Q$, буква за четене (или ϵ) $a \in \Sigma_\epsilon$ и връх на стека $A \in \Gamma$, стойността $\Delta(q, a, A)$ е множество от двойки от вида (p, β), където p е новото състояние и β е думата, която замества A във върха на стека.

Въвели сме ограничение за думата β да има дължина най-много 2.

В много учебници няма такова ограничение, т.е. Δ функцията има сигнатура $\Delta : Q \times \Sigma_\epsilon \times \Gamma \to \mathcal{P}(Q \times \Gamma^\star)$. Има и други вариации на понятието стеков автомат, но всички те се оказват еквивалентни помежду си.

За да опишем формално работата на автомата P дефинираме понятието *конфигурация*: наредена тройка (q, α, γ), където $q \in Q$ е текущото състояние, $\alpha \in \Sigma^\star$ е частта от входа, която остава да се прочете и $\gamma \in \Gamma^\star$ е съдържанието на стека.

Ако стековият автомат P се намира в една конфигурация κ, той може да премине в конфигурация κ' по един от следните два начина:

1. С четене на вход (нормален преход). Ако $\kappa = (q, b\alpha, A\gamma)$, където $b \in \Sigma$ и $A \in \Gamma$, то $\kappa' = (p, \alpha, \beta\gamma)$, където $(p, \beta) \in \Delta(q, b, A)$.

2. Без четене на вход (ϵ-преход). Ако $\kappa = (q, \alpha, A\gamma)$, където $A \in \Gamma$, то $\kappa' = (p, \alpha, \beta\gamma)$, където $(p, \beta) \in \Delta(q, \epsilon, A)$.

Това дефинира релацията $\kappa \vdash_P \kappa'$ на *едностъпков преход*. Често ще обединяваме двата вида преходи в един като считаме, че $b \in \Sigma_\epsilon$.

Релациите $\kappa \vdash_P^\ell \kappa'$ и $\kappa \vdash_P^\star \kappa'$ се дефинират по обичайния начин:

$$\kappa \vdash_P^0 \kappa, \quad \{\ \kappa \vdash_P \kappa'',\ \kappa'' \vdash_P^\ell \kappa'\ \} \rightsquigarrow \kappa \vdash_P^{\ell+1} \kappa',$$

$$\kappa \vdash_P^\star \kappa' \ \Leftrightarrow\ \exists \ell \in \mathbb{N}\ (\kappa \vdash_P^\ell \kappa').$$

Напомняме, че $\vdash_P^\star$ е рефлексивното и транзитивно затваряне на $\vdash_P$.

Дефиниция 7 *Езикът $\mathcal{L}(P)$, който се разпознава от стековия автомат P дефинираме по следния начин:*

$$\mathcal{L}(P) \stackrel{деф}{=} \{\ \alpha \in \Sigma^\star \mid (q_{start}, \alpha, \sharp) \vdash_P^\star (q_{accept}, \epsilon, \epsilon)\ \}.$$

Началната конфигурация винаги е $(q_{\texttt{start}}, \alpha, \sharp)$, където α е входната дума. Заключителната конфигурация е $(q_{\texttt{accept}}, \epsilon, \epsilon)$. По този начин, единственият начин P да приключи успешно едно изчисление е да стигне до състоянието $q_{\texttt{accept}}$, като няма повече вход за четене и съдържанието на стека е празната дума. Забележете, че имаме два вида конфигурации, в които изчислението приключва неуспешно: (1) съдържанието на стека е ϵ, но има още вход за четене или състоянието не е $q_{\texttt{accept}}$; (2) съдържанието на стека не е ϵ, но съответната стойност на функцията Δ е $\emptyset$, т.е. няма позволен преход.

Тъй като стековият автомат е недетерминиран, неговото изчисление ще има вид на дърво. Поради наличието на ϵ-преходи, това дърво може да е безкрайно (съществена разлика от обикновени автомати). Също така е важно да отбележим, че детерминираните стекови автомати не са еквивалентни на недетерминираните.

Пример. Нека да съставим недетерминиран стеков автомат P, който разпознава езика $L = \{a^n b^n \mid n \in \mathbb{N}\}$. Идеята е да използваме стека, за да запазим a-тата в началото на думата. Това ще позволи да съпоставим последващите b-та на запазените a-та.

Формалното описание на автомата P е следното:

- $Q = \{q, p, f\}$;
- $q_{\texttt{start}} = q$ и $q_{\texttt{accept}} = f$;
- $\Sigma = \{a, b\}$ и $\Gamma = \{\sharp, a\}$;
- Релацията на преходите Δ има следната дефиниция:

  (1) $\Delta(q, \epsilon, \sharp) = \{(f, \epsilon)\}$; // трябва да разпознаем ϵ
  (2) $\Delta(q, a, \sharp) = \{(q, a\sharp)\}$;
  (3) $\Delta(q, a, a) = \{(q, aa)\}$; // трупаме a-та в стека
  (4) $\Delta(q, b, a) = \{(p, \epsilon)\}$; // започваме да четем b-та
  (5) $\Delta(p, b, a) = \{(p, \epsilon)\}$; // чистим a-тата от стека
  (6) $\Delta(p, \epsilon, \sharp) = \{(f, \epsilon)\}$.

Подразбира се, че за всички останали тройки (r, x, y), $\Delta(r, x, y) = \emptyset$.

Можем да опишем работата на P по следния начин: от стартовото състояние q започваме да четем a-та от входната дума и ги запазваме в стека (преходи 2 и 3). Когато стигнем до първото b във входа, премахваме едно a от стека и минаваме в състояние p (преход 4). След това продължаваме да четем b-та и за всяко прочетено b, махаме по едно a от стека (преход 5). Изчислението ще е успешно точно когато натрупаните a-та в стека и прочетените b-та се изчерпят едновременно. Тогава стекът съдържа само $\sharp$ и няма повече букви за четене, така че преминаваме в заключителното състояние f (преход 6). Освен това, имаме преход 1, който позволява да разпознаем ϵ. Ако премахнем ϵ от езика (и съответно преход 1), то стековият автомат ще стане детерминиран. Да разгледаме някои примерни изпълнения.

1. При входна дума $\alpha = aabb$:

$$(q, aabb, \sharp) \vdash (f, aabb, \epsilon) \text{ X}$$
$$\top$$
$$(q, abb, a\sharp) \vdash (q, bb, aa\sharp) \vdash (p, b, a\sharp) \vdash (p, \epsilon, \sharp) \vdash (f, \epsilon, \epsilon)$$

Изчислението има един неуспешен и един успешен клон, затова $aabb \in \mathcal{L}(P)$.

2. При входна дума $\alpha = abab$:

$$(q, abab, \sharp) \vdash (f, abab, \epsilon) \text{ X}$$
$$\top$$
$$(q, bab, a\sharp) \vdash (p, ab, \sharp) \vdash (f, ab, \epsilon) \text{ X}$$

Изчислението има два неуспешни клона, затова $abab \notin \mathcal{L}(P)$.

За домашно опишете изпълненията на P при входни думи:
$\alpha = aaabbb$, $\alpha = abbb$, $\alpha = aaab$.

Материалът за десетата лекция от записките:
глава 3.8 (**Алгоритми**), страници 129-133, 135-136,
глава 3.10 (**Недетерминирани стекови автомати**), страници 139-140.

За допълнително четене от учебника на Hopcroft, Motwani, Ullman:
глава 6.1 (Definition of the Pushdown Automaton),
глава 7.1 (Normal Forms for Context-Free Grammars).